

ROS2 Day4 과제 2 보고서

Lifecycle Node + QoS Publisher / Subscriber 구현

학번: 2026406076

이름: 정창규

제출일: 2026-09-14

목차

1. 과제 개요
2. 패키지 구성
3. Lifecycle Publisher 구현
4. Lifecycle Subscriber 구현
5. QoS 설정
6. Manual Launch 구성
7. Auto Launch 구성
8. 실행 결과
9. 느낀 점

1. 과제 개요

이번 과제에서는 ROS2 Lifecycle Node와 QoS를 적용한 Publisher와 Subscriber 패키지를 구현하였다.

Publisher와 Subscriber는 Lifecycle 상태에 따라 동작하도록 구성하였으며, on_configure() 단계에서 QoS 설정과 통신 자원을 준비하고 Active 상태에서 실제 Publish와 Subscribe가 수행되도록 하였다.

또한 Lifecycle 상태를 직접 명령어로 변경하는 Manual Launch와, 실행 후 자동으로 상태가 전환되는 Auto Launch를 각각 작성하였다. 이를 통해 수동 상태 제어와 자동 상태 전환의 차이를 확인하였다.

2. 패키지 구성

이번 과제에서는 changgyu_lifecycle_qos 패키지를 생성하고, Lifecycle Publisher와 Subscriber를 각 별도의 소스 파일로 작성하였다.

패키지 내부에는 src 폴더에 Publisher와 Subscriber 소스 파일을 두었고, launch 폴더에는 Manual Launch와 Auto Launch 파일을 작성하였다. 또한 CMakeLists.txt와 package.xml에서 필요한 의존성과 실행 파일을 등록하였다.

구성한 주요 파일은 다음과 같다.

```
changgyu_lifecycle_qos
├── src
│   ├── lifecycle_publisher.cpp
│   └── lifecycle_subscriber.cpp
├── launch
│   ├── manual_launch.py
│   └── auto_launch.py
├── CMakeLists.txt
└── package.xml
```

각 파일의 역할을 분리하여 Publisher와 Subscriber의 기능, 상태 전환 방식, Launch 실행 방식을 확인하기 쉽게 구성하였다.

3. Lifecycle Publisher 구현

Publisher는 rclcpp_lifecycle::LifecycleNode를 상속하여 작성하였다. 일반 Node와 다르게 상태 전환에 따라 동작이 달라지도록 구성하였다.

on_configure()에서는 QoS를 설정하고 robot_status Topic을 발행하는 Publisher를 생성하였다. Timer도 함께 생성했지만 바로 cancel()하여 아직 메시지를 보내지 않도록 하였다. 따라서 configure가 끝난 뒤 Inactive 상태에서는 Publisher와 Timer가 준비되어 있지만 실제 Publish는 수행되지 않는다.

on_activate()가 호출되면 Publisher를 활성화하고 Timer를 다시 시작하였다. 이후 1초마다 Robot is running 문자열을 발행하도록 구성하였다.

반대로 on_deactivate()에서는 Timer를 멈추고 Publisher를 비활성화하였으며, on_cleanup()에서는 생성한 Timer와 Publisher를 해제하도록 하였다. 이를 통해 Lifecycle 상태에 따라 실제 동작과 자원 정리를 구분할 수 있도록 구현하였다.

```
CallbackReturn on_activate(const rclcpp_lifecycle::State &) override
{
    publisher_>on_activate();
    timer_>reset();

    RCLCPP_INFO(this->get_logger(), "Activated");

    return CallbackReturn::SUCCESS;
}
```

4. Lifecycle Subscriber 구현

Subscriber 역시 rclcpp_lifecycle::LifecycleNode를 상속하여 작성하였다. Publisher가 발행하는 robot_status Topic을 구독하도록 구성하였다.

on_configure()에서는 Publisher와 동일한 QoS 설정을 적용한 뒤 Subscriber를 생성하였다. 하지만 configure가 끝난 직후에는 active_ 값이 false이기 때문에 메시지가 들어오더라도 실제 출력은 수행하지 않도록 하였다. 이를 통해 Inactive 상태에서는 통신 자원만 준비하고 실제 동작은 하지 않도록 구성하였다.

on_activate()가 호출되면 active_ 값을 true로 변경하여 수신한 메시지를 출력하도록 하였다. 반대로 on_deactivate()에서는 다시 false로 변경하여 메시지 출력이 중지되도록 하였다.

on_cleanup()에서는 Subscriber 객체를 해제하여 configure 단계에서 생성한 자원을 정리하도록 하였다. 이를 통해 Publisher와 마찬가지로 Lifecycle 상태에 따라 준비, 실행, 중지, 정리 과정을 구분하였다.

Publisher와 Subscriber는 Active 상태가 아닐 때 실제 동작을 수행하지 않도록 구성하였다.

Publisher는 Inactive 상태에서 Publisher와 Timer가 준비되어 있지만 Timer가 정지되어 있기 때문에 메시지를 발행하지 않는다. Subscriber는 Subscription이 생성되어 있어도 active_ 값이 false이므로 수신한 메시지를 출력하지 않는다. 두 노드가 Active 상태로 전환된 이후에만 메시지 발행과 수신 처리가 실제로 수행된다.

```
CallbackReturn on_activate(const rclcpp_lifecycle::State &) override
{
    active_ = true;

    RCLCPP_INFO(this->get_logger(), "Activated");

    return CallbackReturn::SUCCESS;
}
```

5. QoS 설정

Publisher와 Subscriber에는 동일한 QoS 설정을 적용하였다. 이번 구현에서는 Keep Last, Depth 10, Reliable, Transient Local을 사용하였다.

Keep Last는 최근 메시지만 보관하는 방식이며, Depth 10은 최근 10개의 Sample을 Queue에 유지하도록 설정한 것이다. 이번 구현에서는 오래된 데이터를 계속 보관하기보다 최근 상태 메시지를 일정 개수만 유지하기 위해 Depth를 10으로 설정하였다.

Reliable은 메시지가 손실되었을 때 재전송을 시도하는 방식이다. 이번 과제에서는 로봇의 상태 정보를 전달하기 때문에 메시지가 누락되는 것보다 안정적으로 전달되는 것이 중요하다고 판단하여 Reliable을 선택하였다.

또한 Transient Local을 사용하여 Subscriber가 Publisher보다 늦게 실행되더라도 Publisher가 보관하고 있는 최근 메시지를 받을 수 있도록 하였다. 이를 통해 Subscriber가 늦게 연결되는 경우에도 현재 로봇의 상태 정보를 확인할 수 있도록 구성하였다.

Publisher와 Subscriber에는 동일한 QoS 설정을 적용하여 통신 호환성을 맞추었다. Reliable Publisher와 Reliable Subscriber, Transient Local Publisher와 Transient Local Subscriber는 서로 호환되는 조합이다.

```
rclcpp::QoS qos(rclcpp::KeepLast(10));
qos.reliable();
qos.transient_local();
```

6. Manual Launch 구성

Manual Launch 파일은 Lifecycle Publisher와 Subscriber를 실행하되, 상태 전환은 자동으로 수행하지 않도록 작성하였다.

Launch 파일을 실행하면 두 노드는 처음에 Unconfigured 상태로 시작한다. 이후 사용자가 `ros2 lifecycle set` 명령어를 직접 입력하여 `configure`, `activate` 순서로 상태를 변경하도록 하였다. 이를 통해 각 상태가 실제로 어떻게 전환되는지 확인할 수 있었다.

수동 제어 방식에서는 노드의 현재 상태를 직접 확인하면서 필요한 시점에 상태를 변경할 수 있기 때문에 Lifecycle의 동작 과정을 확인하기에 적합하였다.

```
publisher = LifecycleNode(
    package='changgyu_lifecycle_qos',
    executable='lifecycle_publisher',
    name='lifecycle_publisher',
    namespace='',
    output='screen'
)
```

```
ros2 lifecycle set /lifecycle_publisher configure
ros2 lifecycle set /lifecycle_publisher activate
```

Lifecycle 상태를 수동으로 변경하는 명령어

7. Auto Launch 구성

Auto Launch 파일에서는 Lifecycle Publisher와 Subscriber를 실행한 뒤 상태가 자동으로 전환되도록 구성하였다.

두 노드에는 autostart=True를 설정하였다. 이를 통해 Launch 파일이 실행되면 사용자가 별도의 ros2 lifecycle set 명령을 입력하지 않아도 configure와 activate 과정이 자동으로 수행되어 Active 상태까지 전환된다.

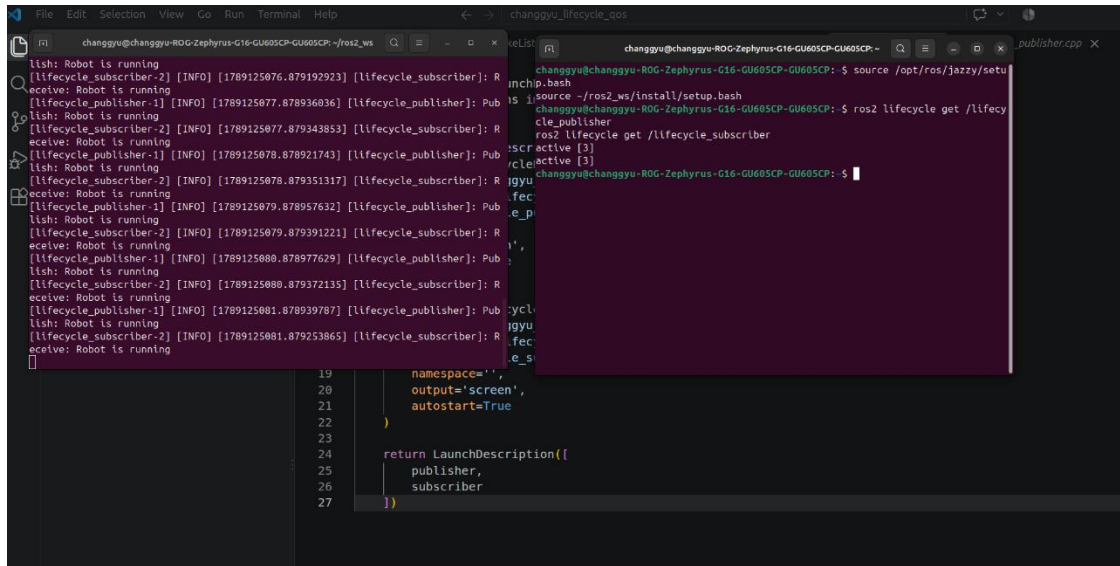
수동 방식과 달리 상태 전환 명령을 직접 입력하지 않아도 되기 때문에 여러 Lifecycle Node를 한 번에 실행해야 하는 경우에 편리하게 사용할 수 있다. 과제에서는 수동 제어 Launch와 자동 상태 전환 Launch를 각각 작성하도록 요구하고 있다.

```
publisher = LifecycleNode(
    package='changgyu_lifecycle_qos',
    executable='lifecycle_publisher',
    name='lifecycle_publisher',
    namespace='',
    output='screen',
    autostart=True
)
```

8. 실행 결과

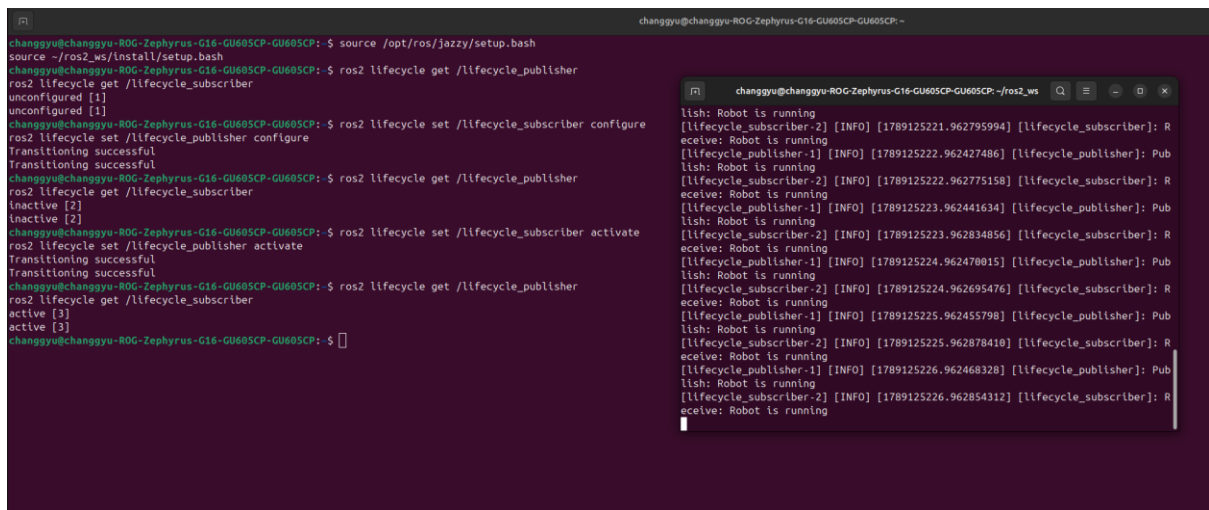
먼저 Manual Launch를 실행한 뒤 Publisher와 Subscriber의 초기 상태를 확인하였다. 두 노드 모두 처음에는 unconfigured [1] 상태로 시작하였으며, 이후 configure 명령을 입력하자 inactive [2] 상태로 전환되었다.

그다음 activate 명령을 직접 입력하여 두 노드를 active [3] 상태로 전환하였다. Active 상태가 된 이후 Publisher에서는 Robot is running 메시지를 발행하였고, Subscriber에서도 같은 메시지를 정상적으로 수신하는 것을 확인하였다.



Auto Launch에서는 별도의 상태 전환 명령을 입력하지 않고 Launch 파일만 실행하였다. 실행 후 Publisher와 Subscriber가 자동으로 active [3] 상태까지 전환되었으며, 바로 Publish와 Subscribe가 수행되는 것을 확인하였다.

이를 통해 Manual Launch에서는 사용자가 상태를 직접 변경해야 하고, Auto Launch에서는 상태 전환 과정이 자동으로 처리된다는 차이를 확인할 수 있었다. 과제에서 요구한 수동 제어 Launch와 자동 상태 전환 Launch가 각각 정상적으로 동작하였다.



9. 느낀 점

이번 과제를 진행하면서 Lifecycle Node가 단순히 상태 이름을 나누는 기능이 아니라, 실제 동작 시작 시점을 제어하기 위한 구조라는 것을 이해할 수 있었다.

특히 Manual Launch를 이용하여 Unconfigured → Inactive → Active 순서로 직접 상태를 변경해보면서 각 단계의 차이를 확실하게 확인할 수 있었다. Inactive 상태에서는 준비만 되어 있고, Active가 된 이후에 실제 Publish와 Subscribe가 수행된다는 점이 가장 이해하기 쉬웠다.

또한 QoS 설정을 코드에 직접 적용하면서 Publisher와 Subscriber가 같은 Topic을 사용하더라도 통신 설정이 중요하다는 것을 알게 되었다.

마지막으로 Manual Launch와 Auto Launch를 각각 작성해보면서 같은 Lifecycle Node라도 실행 방법에 따라 상태 전환을 직접 제어하거나 자동으로 처리할 수 있다는 차이를 확인하였다. 여러 노드를 함께 사용하는 ROS2 시스템에서는 상황에 맞는 상태 관리 방식이 필요하다고 생각하였다.